

Distributed Systems

Lecture23: Consensus

Mohammad Reza Shakournia

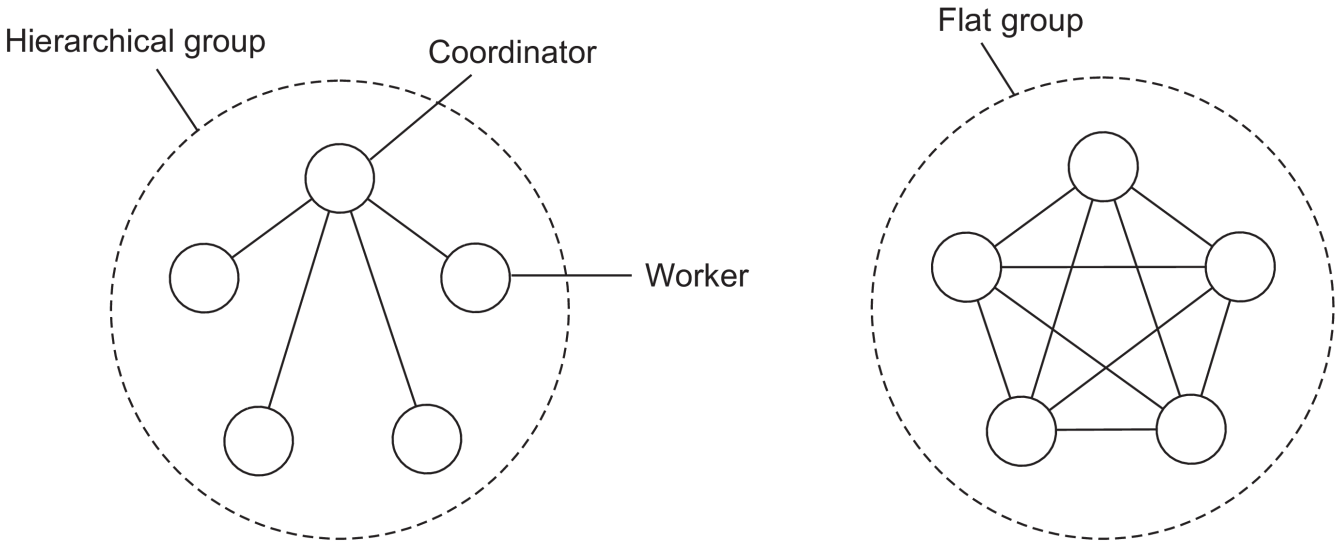
shakournia@ut.ac.ir

School of Electrical and Computer Engineering,
University of Tehran,
Tehran, Iran

Process Resilience

- **Basic**

  Protect against malfunctioning processes through process replication, organizing multiple processes into a process group. Distinguish between flat groups and hierarchical groups.in every one. Computationally cheap.



Groups and Failure Masking

- **k-fault tolerant group**

  When a group can mask any k concurrent member failures (k is called degree of fault tolerance).
- **How large does a k-fault tolerant group need to be?**
  - With halting failures (crash/omission/timing failures): we need a total of k +1 members as no member will produce an incorrect result, so the result of one member is good enough.
  - With arbitrary failures: we need 2k +1 members so that the correct result can be obtained through a majority vote.
- **Important assumptions**
  - All members are identical
  - All members process commands in the same order
- Result: We can now be sure that all processes do exactly the same thing.

Consensus

- **Prerequisite**

  In a fault-tolerant process group, each nonfaulty process executes the same commands, and in the same order, as every other nonfaulty process.

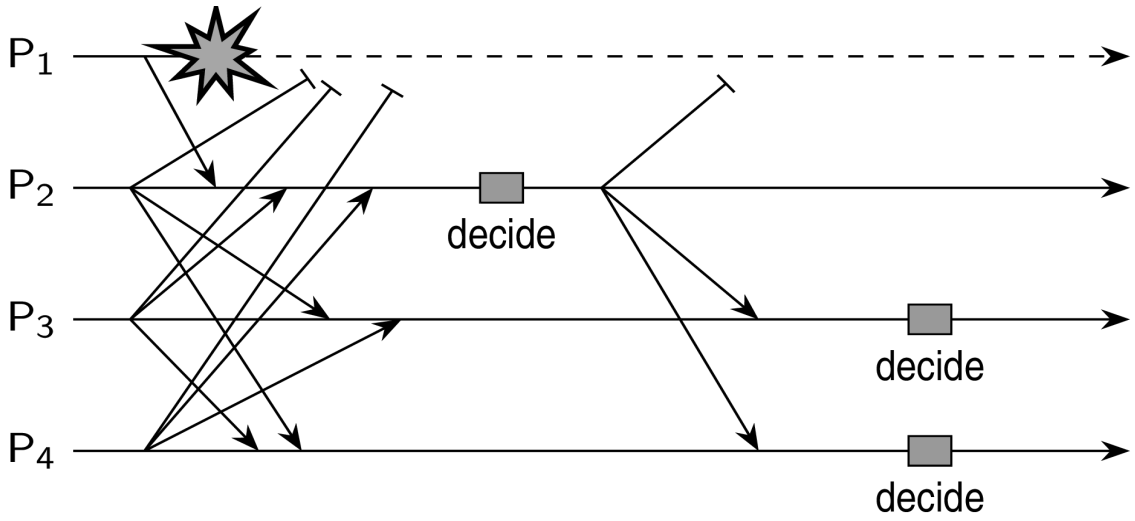
- **Reformulation**

  Nonfaulty group members need to reach consensus on which command to execute next.

Flooding-based Consensus

- **System model**
  - A process group **P** = {P_1, . . . ,P_n}
  - Fail-stop failure semantics, i.e., with reliable failure detection
  - A client contacts a P_i requesting it to execute a command
  - Every P_i maintains a list of proposed commands
- **Basic algorithm (based on rounds)**
  1. In round r , P_i multicasts its known set of commands $\mathbf{C}_{ri}$ to all others
  2. At the end of r , each P_i merges all received commands into a new $\mathbf{C}_{r+1\,i}$.
  3. Next command cmd_i selected through a globally shared, deterministic function: $cmd_i \leftarrow select(\mathbf{C}_{r+1i})$.

Flooding-based Consensus: Example



- **Observations**
  - P2 received all proposed commands from all other processes ⇒ makes decision.
  - P3 may have detected that P1 crashed, but does not know if P2 received anything, i.e., P3 cannot know if it has the same information as P2 ⇒ cannot make decision (same for P4).

Raft

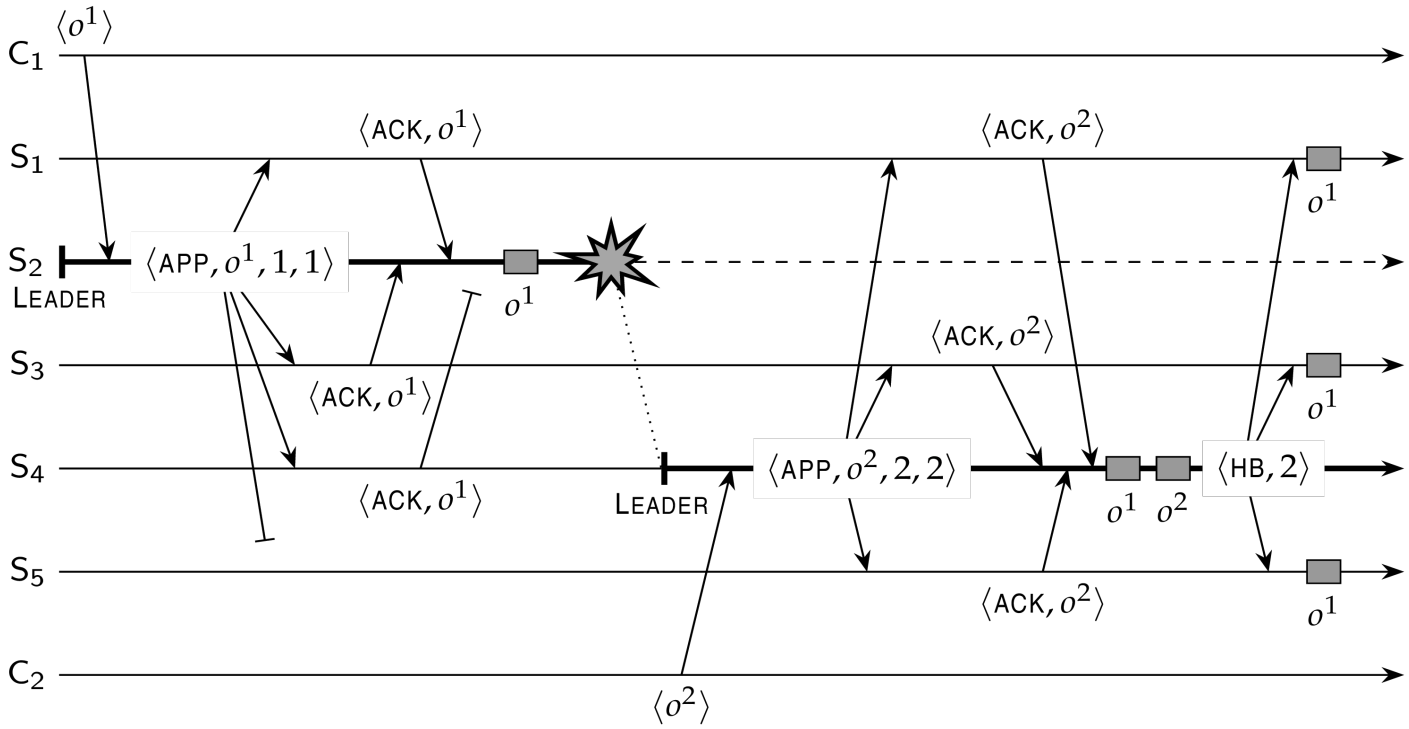
- **Developed for understandability**
  - Uses a fairly straightforward leader-election algorithm. The current leader operates during the current term.
  - Every server (typically, five) keeps a log of operations, some of which have been committed. A backup will not vote for a new leader if its own log is more up to date.
  - All committed operations have the same position in the log of each respective server.
  - The leader decides which pending operation is to be committed next ⇒ a primary-backup approach.

Raft

- **When submitting an operation**
  - A client submits a request for operation o.
  - The leader appends the request ⟨o, t,n+1⟩ to its own log (registering the current term t and length n+1 of its log).
  - The log is (conceptually) broadcast to the other servers.
  - The others (conceptually) copy the log and acknowledge the receipt.
  - When a majority of acks arrives, the leader commits o.
- **Note**

  In practice, only updates are broadcast. At the end, every server has the same view and knows about the c committed operations. Note that effectively, any information at the backups is overwritten.

Raft: When a Leader Crashes



- **Crucial observations**
  - The new leader has the most committed operations in its log.
  - Any missing commits will eventually be sent to the other backups.

Realistic Consensus: Paxos

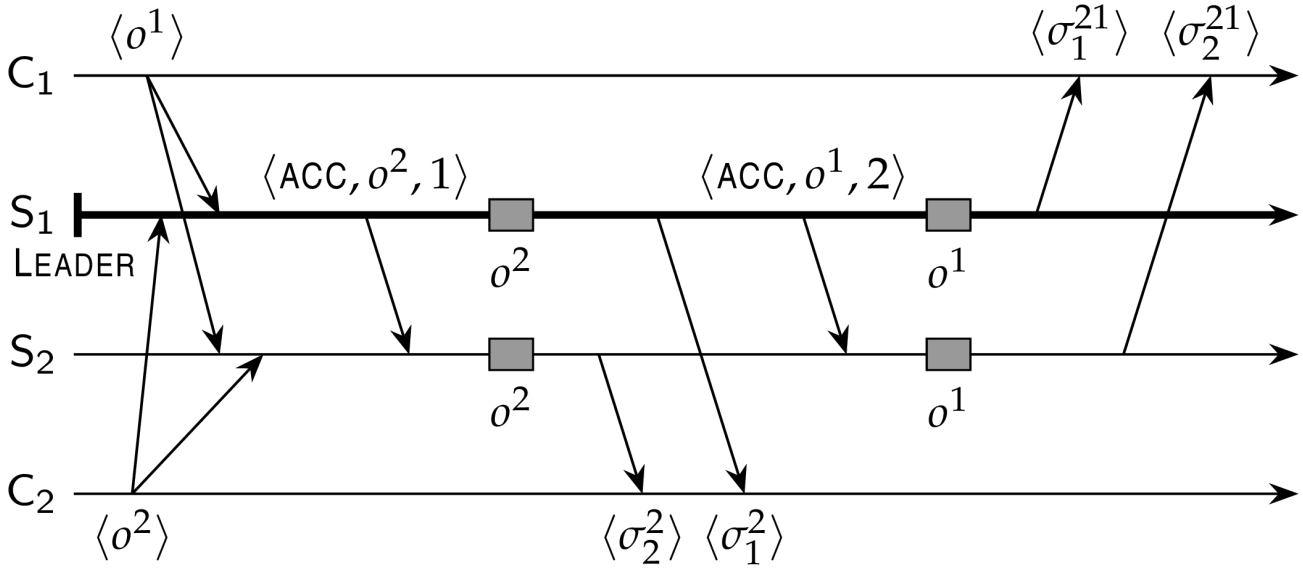
- **Assumptions (rather weak ones, and realistic)**
  - A partially synchronous system (in fact, it may even be asynchronous).
  - Communication between processes may be unreliable: messages may be lost, duplicated, or reordered.
  - Corrupted message can be detected (and thus subsequently ignored).
  - All operations are deterministic: once an execution is started, it is known exactly what it will do.
  - Processes may exhibit crash failures, but not arbitrary failures.
  - Processes do not collude.
- **Understanding Paxos**

  We will build up Paxos from scratch to understand where many consensus algorithms actually come from.

Paxos Essentials

- **Starting Point**
  - We assume a client-server configuration, with initially one primary server.
  - To make the server more robust, we start with adding a backup server.
  - To ensure that all commands are executed in the same order at both servers, the primary assigns unique sequence numbers to all commands. In Paxos, the primary is called the leader.
  - Assume that actual commands can always be restored (either from clients or servers) ⇒ we consider only control messages.

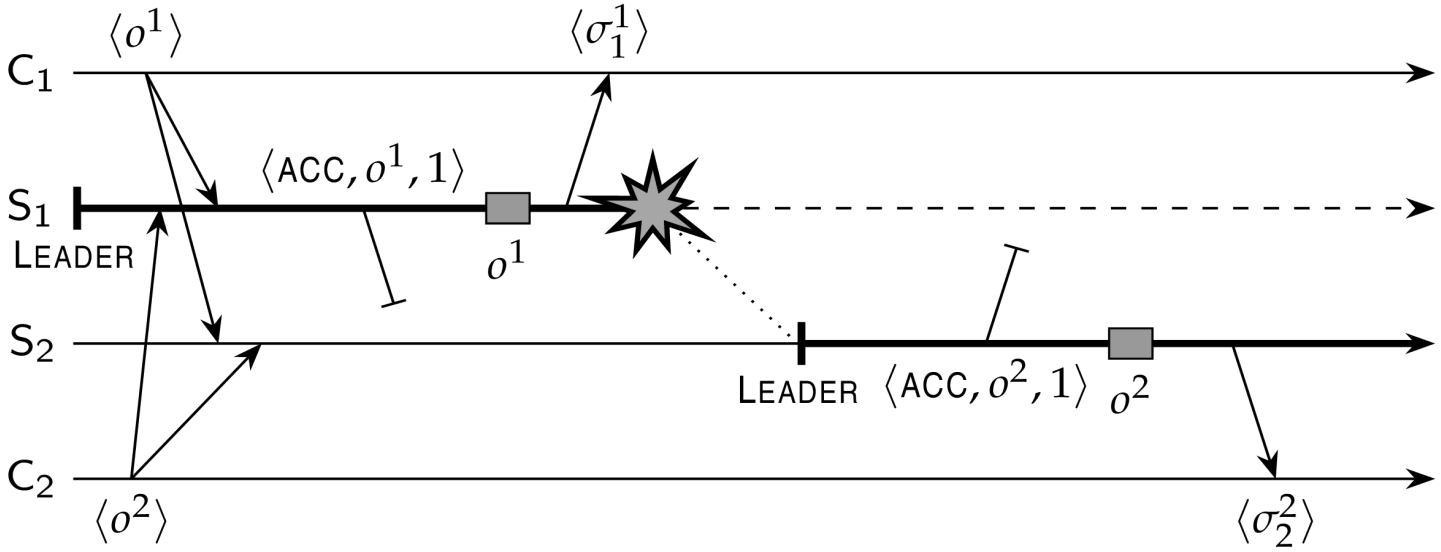
Two-server Situation



Handling Lost Messages

- **Some Paxos terminology**
  - The leader sends an accept message ACCEPT(o, t) to backups when assigning a timestamp t to command o.
  - A backup responds by sending a learn message: LEARN(o, t)
  - When the leader notices that operation o has not yet been learned, it retransmits ACCEPT(o, t) with the original timestamp.

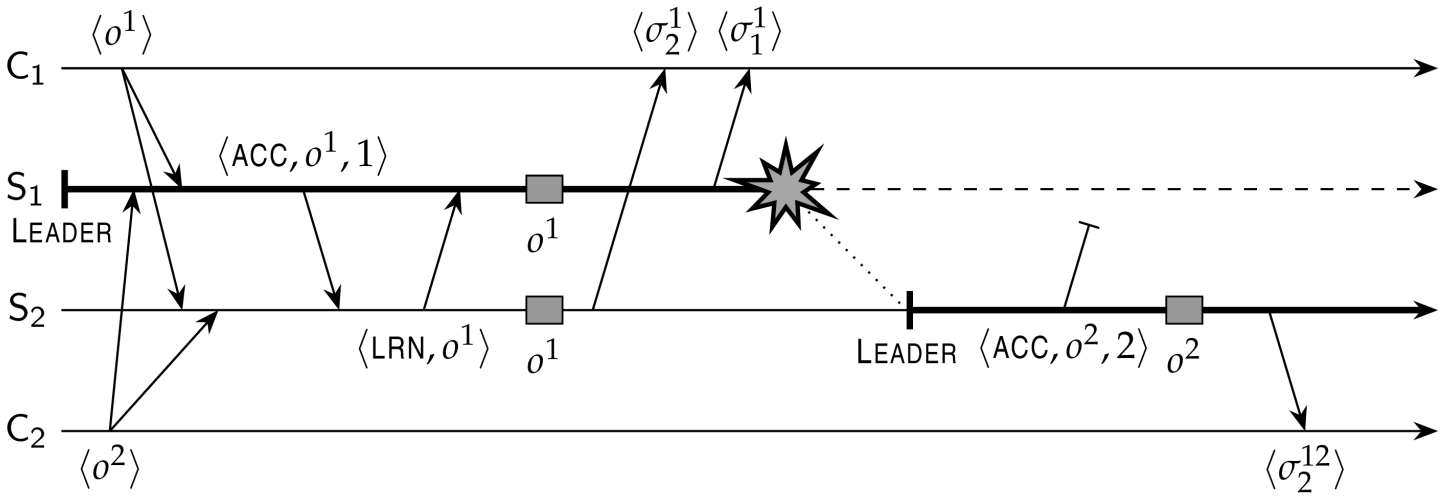
Two Servers and One Crash: Problem



- **Problem**

  Primary crashes after executing an operation, but the backup never received the accept message.

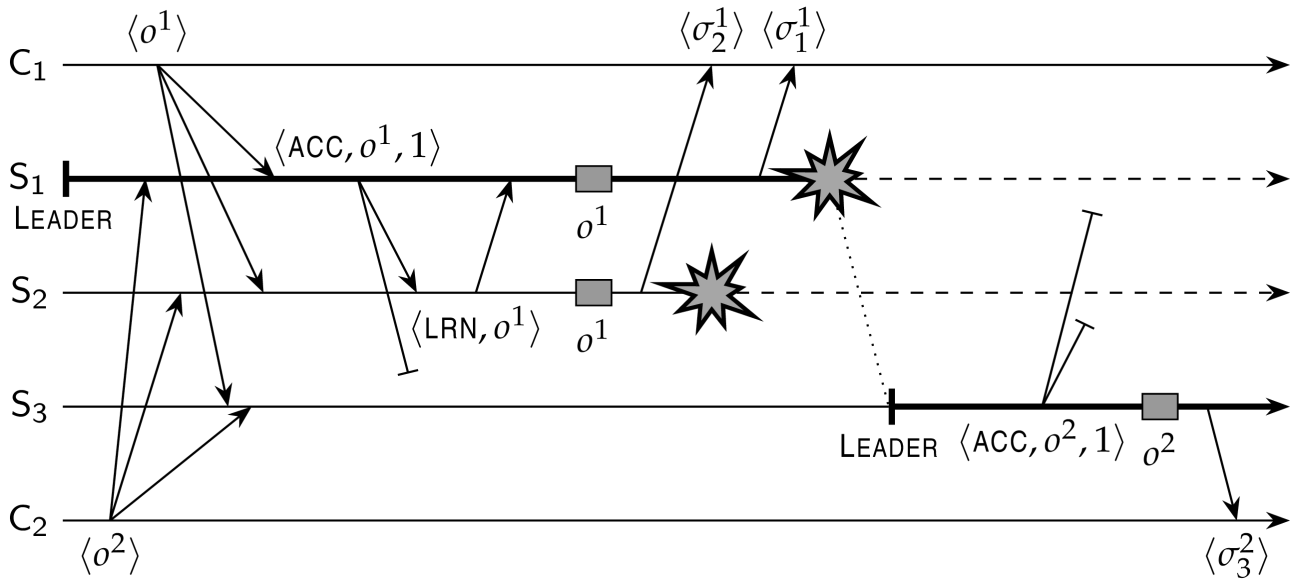
Two Servers and One Crash: Solution



- **Solution**

  Never execute an operation before it is clear that is has been learned.

Three Servers and Two Crashes: Still a Problem?



- **Scenario**

  What happens when LEARN(o1) as sent by S2 to S1 is lost?

- **Solution**

  S2 will also have to wait until it knows that S3 has learned o1.

Paxos: Fundamental Rule

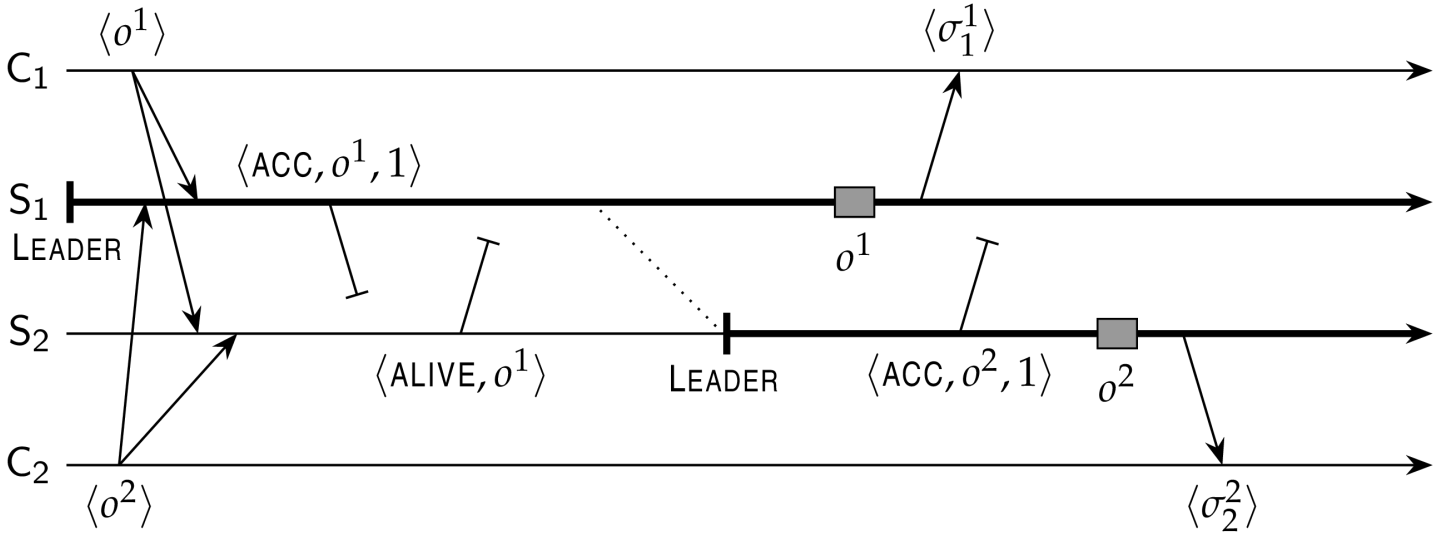
- **General rule**

  In Paxos, a server S cannot execute an operation o until it has received a LEARN(o) from all other nonfaulty servers.

Failure Detection

- **Practice**

  Reliable failure detection is practically impossible. A solution is to set timeouts, but take into account that a detected failure may be false.



Required Number of Servers

- **Observation**

  Paxos needs at least three servers

- **Adapted fundamental rule**

  In Paxos with three servers, a server S cannot execute an operation o until it has received at least one (other) LEARN(o) message, so that it knows that a majority of servers will execute o.

Required Number of Servers

- **Assumptions before taking the next steps**
  - Initially, S1 is the leader.
  - A server can reliably detect it has missed a message, and recover from that miss.
  - When a new leader needs to be elected, the remaining servers follow a strictly deterministic algorithm, such as S1 → S2 → S3.
  - A client cannot be asked to help the servers to resolve a situation.
- **Observation**

  If either one of the backups (S2 or S3) crashes, Paxos will behave correctly:operations at nonfaulty servers are executed in the same order.

Leader Crashes After Executing O^1

- **S_3 is completely ignorant of any activity by S_1**
  - S_2 received ACCEPT$(o, 1)$, detects crash, and becomes leader.
  - S_3 even never received ACCEPT$(o, 1)$.
  - If S_2 sends ACCEPT$(o^2, 2) \Rightarrow S_3$ sees unexpected timestamp and tells S_2 that it missed o^1.
  - S_2 retransmits ACCEPT$(o^1, 1)$, allowing S_3 to catch up.
- **S_2 missed ACCEPT$(o^1, 1)$**
  - S_2 did detect crash and became new leader.
  - If S_2 sends ACCEPT$(o^1, 1) \Rightarrow S_3$ retransmits LEARN(o^1).
  - If S_2 sends ACCEPT$(o^2, 1) \Rightarrow S_3$ tells S_2 that it apparently missed ACCEPT$(o^1, 1)$ from S_1, so that S_2 can catch up.

Leader Crashes After Sending ACCEPT($o^1, 1$)

- **S3 is completely ignorant of any activity by S_1**

  As soon as S_2 announces that o^2 is to be accepted, S_3 will notice that it missed an operation and can ask S_2 to help recover.

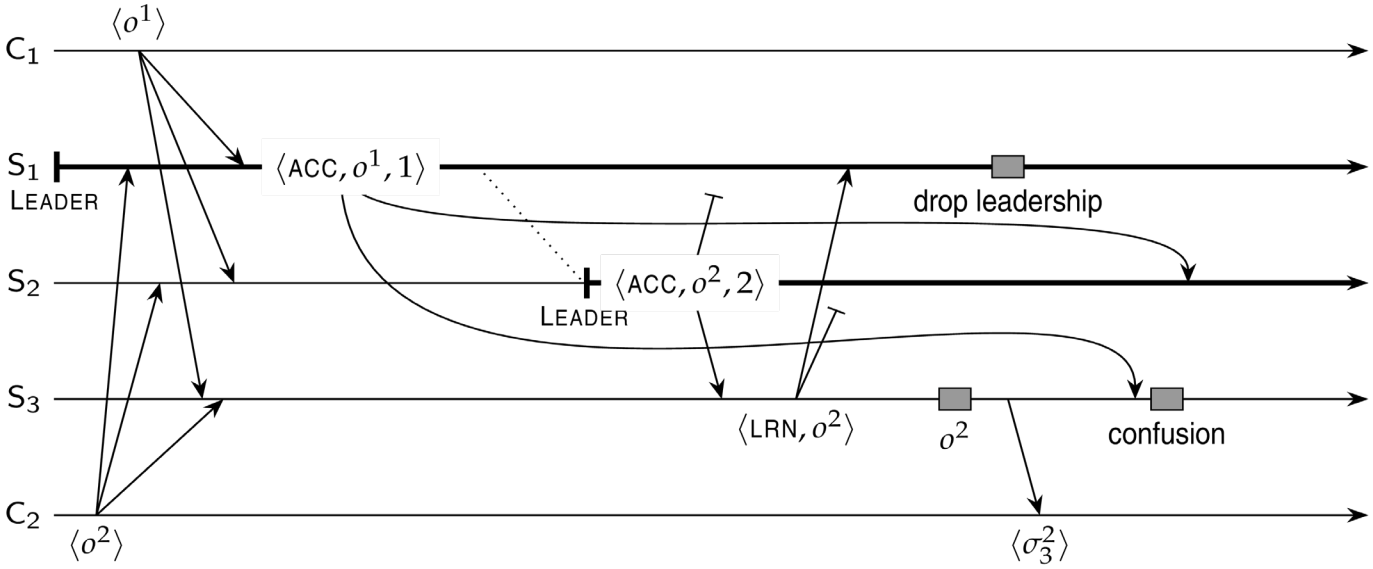
- **S_2 had missed ACCEPT($o^1, 1$)**

  As soon as S_2 proposes an operation, it will be using a stale timestamp, allowing S_3 to tell S_2 that it missed operation o^1.

- **Observation**

  Paxos (with three servers) behaves correctly when a single server crashes, regardless when that crash took place

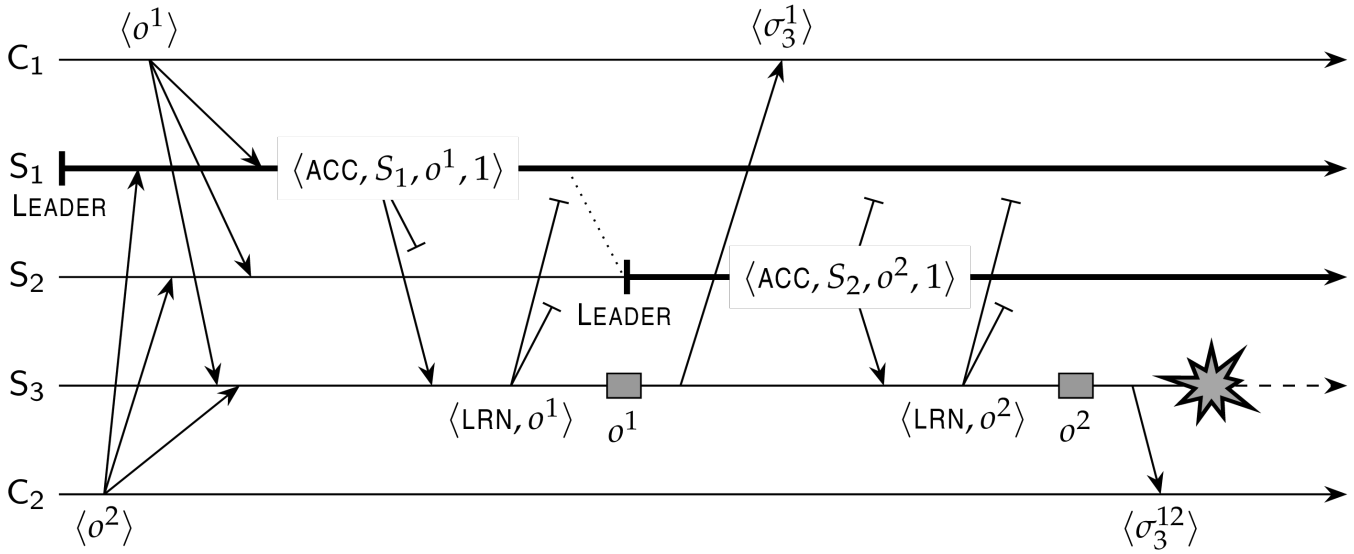
False Crash Detections



- **Problem and Solution**

  *S*3 receives ACCEPT$(o^1, 1)$, but much later than ACCEPT$(o^2, 1)$. If it knew who the current leader was, it could safely reject the delayed accept message ⇒ leaders should include their ID in messages.

But What About Progress?



- **Essence of solution**

  When S_2 takes over, it needs to make sure that any outstanding operations initiated by S_1 have been properly **flushed**, i.e., executed by enough servers. This requires an **explicit leadership takeover** by which other servers are informed before sending out new accept messages.